

Introduction

Scanning a large filesystem for old files can be accelerated by distributing the work across multiple Python containers orchestrated with Kubernetes. The solution involves a **master-worker architecture** where a parent container (master) scans a top-level directory and launches child containers (workers) for subdirectories. This design enables **parallel and recursive scanning** of the directory tree. The scanner will identify files older than a given age threshold and report these in real time via a messaging system. We must also accommodate both **on-demand triggers** and **scheduled runs** (e.g. using Kubernetes CronJobs for periodic scans). In the following sections, we outline the architecture, design patterns, containerization techniques, messaging setup, result aggregation, and best practices (error handling, security, logging, monitoring) for a production-grade implementation.

Architecture Overview

The scanning system is structured as a distributed workflow on Kubernetes, using Jobs and possibly sidecar containers for supporting tasks. At a high level, a **parent Job** acts as the orchestrator: it scans the root directory, discovers subdirectories, and spawns a new Kubernetes Job for each subdirectory. Each **child Job** runs a Python container to scan its assigned sub-tree, possibly spawning further jobs for nested subdirectories if deep recursion is needed. All scanning containers send progress updates and results (e.g. lists of “old” files beyond the age threshold) to a central message queue in real time. A separate aggregator service consumes these messages to provide a unified live view of progress and findings. Scheduled execution is handled via a Kubernetes CronJob that periodically launches the parent Job, while on-demand scans can be triggered by an API or CLI that creates the parent Job dynamically.

Overall architecture: A CronJob (for scheduled runs) or external trigger launches a parent scanner Job Pod, which enumerates directories and spawns child Jobs for each subdirectory. Each child scanning Pod reports progress and results to a message queue (e.g. Kafka, RabbitMQ, or Redis Streams). An aggregator service consumes the queue to update a live dashboard or store results in a database for visualization.*

Distributed Scanning Workflow on Kubernetes

Master-Worker Job Pattern: The parent scanner (master) is a Python process running in a Kubernetes Job Pod that coordinates the work. It starts by scanning the top-level directory to identify immediate subdirectories and files. For each subdirectory, it uses the Kubernetes API (via the Python Kubernetes client) to create a new Job object, passing the subdirectory path as a parameter/env var to the child job. This effectively “spawns” a child Pod for each subdirectory in parallel. The parent can either spawn jobs for *all* nested directories recursively, or handle recursion in stages (parent -> children -> grandchildren, etc.). A simpler approach is to let each child job scan its entire sub-tree (including any deeper subdirectories sequentially or with threads) so that the spawning is only one level deep – this avoids an explosion of too many tiny jobs if the directory tree is very deep. The **Kubernetes Jobs API** supports this fan-out model; for example, one can create multiple Jobs from a template to process items in parallel ¹ ² . Each Job runs to completion, scanning its directory, then exits. Kubernetes ensures that each Job’s Pod eventually completes successfully or fails/retries according to its spec (with configurable back-off and retries).

Work Queue Alternative: Instead of one parent directly spawning N Jobs, another design is to use a *work queue* of directories. The parent job could enqueue subdirectory paths into a message queue, and a pool of worker Pods (spawned via a single Kubernetes Job with a fixed parallelism) can pull tasks from this queue. In Kubernetes' recommended pattern for batch jobs, each worker Pod picks up one unit of work from a queue, processes it, and repeats until the queue is empty ³ ⁴. This approach decouples task distribution from the master and can dynamically scale the number of workers (even via an auto-scaler like KEDA) based on queue backlog. For example, the parent can push all subdirectory paths into a Redis or RabbitMQ queue, and child workers continuously take a directory path, scan it (possibly pushing further subdirs back to the queue), until all done. The choice between *explicit spawning of Jobs per subdirectory* vs. a *central work queue with a fixed worker pool* depends on complexity and scale – the former gives fine-grained control and isolation per subdirectory, while the latter is simpler when tasks are numerous and short-lived.

Sidecar Containers for Progress: In each scanning Pod, we can employ a sidecar container pattern to handle auxiliary tasks like sending progress updates or logs, while the main container focuses on scanning. For example, a sidecar might stream log output or file scan metrics to the message queue in real time, or handle connectivity to the message system if we want to keep the main scanner code lightweight. Each Pod would contain two containers: the **main scanner (Python)** and a **sidecar** for progress reporting. The sidecar could read the filesystem concurrently (or read a shared volume or the main container's logs) to detect when files are processed and then emit messages. Kubernetes supports multiple containers in one pod, but prior to recent Kubernetes versions there was no native awareness of which container is main vs sidecar – they were managed the same, which could lead to lifecycle issues (e.g. a sidecar not terminating when the main container finishes). With Kubernetes v1.28+, however, there is built-in support for **native sidecar containers** that allows you to designate certain containers as sidecars with a lifecycle tied to the main container ⁵ ⁶. This ensures sidecars start before the primary container and don't block pod termination on completion. In practice, if not using the new sidecar feature, one must implement workarounds (like having the main container wait or the sidecar watch for main's termination) to avoid jobs hanging ⁷ ⁸. The new approach simplifies this: by moving supporting containers into the `initContainers` section with `restartPolicy: Always`, Kubernetes will treat them as true sidecars and manage their startup/shutdown order properly ⁹.

Sidecar pattern in a scanning Pod: The Pod running a file scan job can include a secondary "sidecar" container responsible for progress reporting. The main Python scanner container traverses files and identifies old files, while the sidecar container handles streaming updates (via a message queue client or log shipping). Kubernetes's native sidecar support (v1.28+) ensures the sidecar starts before the main process and terminates gracefully after the main container exits ⁵.*

Orchestration Tools: The logic for spawning child jobs can be custom-coded in Python (using the official Kubernetes Python client to create Job objects). Ensure the parent container's service account has permission (RBAC Role) to create jobs in the namespace. An alternative is to use a Kubernetes workflow engine or operator: for example, **Argo Workflows** or **Kubernetes Operators** can manage multi-step scanning pipelines. The open-source project *secureCodeBox* uses a custom Kubernetes Operator to orchestrate security scan jobs in a pipeline (scan -> parse -> persist) with multiple Kubernetes Jobs and sidecars ¹⁰ ¹¹. In our case, a custom operator could watch for a "ScanFolder" CustomResource and then spawn the parent and child jobs accordingly, but this adds complexity. For most cases, a simple parent Job (or even just a Python script outside K8s invoking `kubectl create job` for each subdir) is sufficient.

Recursive Scanning Strategy: The system should handle nested directories recursively. If the approach is *multi-level spawning*, each job (including children) could itself enumerate any subdirectories it finds and spawn further jobs. This can lead to a large number of jobs if the hierarchy is deep, so consider depth-limiting or switching to an internal loop for deeper levels. A safer strategy is that the parent spawns jobs for *first-level subdirectories*, and each child job scans its entire sub-tree internally (using Python recursion or iterative walk) rather than spawning more jobs. This way, parallelism is achieved at the top level, and within each sub-tree the scanning is serialized (or limited parallelism via threads). This approach confines the number of Kubernetes jobs to at most the number of subdirectories at the root level, which could still be large but is manageable with Kubernetes Jobs. If truly massive parallelism is required (e.g. thousands of subfolders), then the work-queue model with auto-scaling workers is more appropriate than thousands of individual Job objects.

Containerization and Performance (Lightweight Python Containers)

Each scanner (parent or child) runs in a Python-based container. To achieve fast startup and low resource usage, follow container best practices:

- **Use a Small Base Image:** Choose a minimal Python base image such as an official `python:3.X-slim` variant or even a distroless Python image. Smaller images mean less data to pull and attack surface is reduced, containing only essential runtime components ¹². For example, the full `python:3.12` image can be >1 GB, whereas `python:3.12-slim` is ~130 MB and Alpine-based images ~50 MB ¹³. The slim Debian-based image is often a good balance between size and compatibility, whereas Alpine is smaller but can have compatibility and build-time tradeoffs (due to needing to compile some Python packages from source) ¹⁴. Minimizing image size helps reduce cold start time (faster to pull and instantiate containers).
- **Multi-stage Build:** Use multi-stage Docker builds to separate the build environment from the runtime environment. For instance, install any needed packages (if compiling native dependencies) in a builder image, then copy only the final artifacts into a clean, slim runtime image. This keeps the runtime image small and free of build tools.
- **Minimize Startup Overhead:** Structure the Python application to load quickly. Avoid importing heavy modules at startup if not necessary. For a simple file scanning script, the imports should be minimal (e.g. use Python's standard `os`, `pathlib`, `datetime` libraries). If using a messaging client (Kafka or RabbitMQ libraries can be somewhat heavy), import and initialize the client lazily (only when needed) to keep initial startup snappy. Ensure that the container doesn't do any lengthy initialization; it should begin scanning almost immediately on launch.
- **Resource Limits:** Right-size the container's resources. Scanning files (getting file metadata and listing directories) is usually I/O-bound on disk. CPU usage is typically low, so we can request low CPU (e.g. 0.1 or 0.2 cores) but perhaps slightly more memory if the directory listings are large (to hold file lists). In testing, profile how much memory is used for scanning a large directory and set a limit accordingly with some headroom. By specifying requests/limits, Kubernetes can better schedule the many scanning pods on the cluster without straining nodes.

- **File Access Optimization:** If the file system being scanned is a network mount (NFS, CephFS, etc.), consider that many scanner pods may hit it concurrently. This could create I/O contention. You might introduce a slight delay or rate-limit in each scanner or limit the number of parallel jobs to avoid overwhelming the storage. Within each container, you can also use multithreading to traverse a directory faster (e.g. spawn a few threads to stat files in parallel) if the directory is huge, but be cautious as Python threads are limited by the GIL for CPU (though for I/O-bound operations like `os.stat`, threads can help overlap I/O latency). Alternatively, Python's `os.walk` with topdown recursion is usually sufficient for most cases.
- **File Age Calculation:** The scanner logic will check file modification times against the threshold (say, "older than X days"). In Python, this can be done by obtaining the file's modification timestamp (e.g. `os.stat(path).st_mtime` or using `pathlib.Path.stat().st_mtime`) and comparing to the current time. Use Python's `datetime` to compute the age in days. Any file exceeding the threshold should be recorded (either in a list to output at end, or reported immediately via the message queue). To avoid huge memory usage, it's wise to stream results out as they are found rather than collecting millions of entries in memory. Each old file found can trigger an event to the message queue (or at least write to stdout which the sidecar/aggregator can capture).
- **Ephemeral Containers:** Remember that these containers are short-lived batch jobs. They should ideally exit cleanly after completing their scan. Ensure the Python script ends with an exit code 0 on success, and non-zero on failure (so Kubernetes can mark the Job failed and retry if configured). Since containers are ephemeral, any state they produce that's needed later (e.g. list of old files) must be externalized (via the message queue or by writing to a shared volume or object storage, though message queue is our chosen method for real-time results). Also use the Kubernetes **TTL controller for finished Jobs** to auto-delete Job objects and pods after a certain time (e.g. set `ttlSecondsAfterFinished` in the Job spec) to avoid accumulation of completed job resources.
- **Dockerfile Best Practices:** General Dockerfile practices apply – e.g. combine commands to reduce layers and image size, use `.dockerignore` to avoid sending unnecessary build context, and use `COPY` instead of `ADD` (unless you need `ADD` functionalities) ¹⁵ ¹⁶ . Also ensure only one process runs per container (the scanner is one process; if we use sidecars, that's separate containers in the pod, not multiple processes in one container) ¹⁷ .
- **Non-Root User:** For security (and potentially to limit damage if misused), run the container as a non-root user. Python apps typically do not need root inside the container. By default, Docker images run as root, which is a bad practice since a process running as root in the container runs as root on the host as well (meaning if compromised, the attacker has root access on the host) ¹⁸ . Create a dedicated user in the Dockerfile (and adjust file permissions if needed) to drop privileges ¹⁹ ²⁰ . One caveat: if the container needs to access a hostPath volume with the files to scan, ensure that user has permissions to read those files (you might use a specific user ID that matches file ownership, or as a fallback you might run as root but with read-only fs access – still not ideal). In general, principle of least privilege is key.

Real-Time Progress Communication (Message Queue)

A central message queue enables all scanning jobs to report their status and findings in real time, and allows a consumer to aggregate these messages for live feedback. Several messaging technologies can meet this requirement:

- **Apache Kafka:** A distributed log system, ideal for high-volume streaming events with durability. Each scanning event (progress update or file-found result) can be published to a Kafka topic. Kafka is scalable and ensures ordered, durable storage of events. For example, a “scan-progress” topic could receive messages like `{"runId": "...", "path": "/dir/subdir", "event": "STARTED"}` or `{"runId": "...", "file": "/dir/subdir/file.txt", "last_modified": "...", "event": "OLD_FILE_FOUND"}`. Consumers can read in real time or later (since Kafka retains messages for a configured period). If using Kafka, consider using a schema (e.g. JSON schema or Avro) for the messages for consistency. The downside is Kafka’s operational complexity – it’s heavier to run on-premises if not already part of your stack.
- **RabbitMQ:** A lightweight message broker using AMQP. RabbitMQ is well-suited for real-time messaging with routing. You could set up an exchange and routing keys for different event types (progress vs results). For instance, a “progress” queue might carry heartbeat messages (like percentage done in a directory) and a “results” queue carries file findings. RabbitMQ ensures delivery (with acknowledgments) and supports consumer groups (competing consumers). It’s simpler to manage than Kafka for moderate throughput. Each scanning container can open a connection (using a Python client like `pika`) and publish messages as it scans.
- **Redis Streams:** If you already have a Redis, its Streams feature can act as a lightweight queue. Scanners can `XADD` events to a stream, and an aggregator can use consumer groups to read them. Redis Streams provide persistence and replayability similar to Kafka (though not as scalable for huge volumes), and are relatively easy to use via the `redis-py` library. They are a good option if the event volume is not extremely high and if low latency is needed with minimal overhead.

When designing the **event schema**, include enough information to trace and aggregate: e.g. a unique run ID for the scan (so multiple concurrent scans or historical scans can be distinguished), perhaps the path of the directory or file the event relates to, a timestamp, and event type (START, PROGRESS, FILE_FOUND, END, ERROR, etc.). For file result events, include file metadata like last modified time or age. For progress events, you might include how many files processed so far out of how many (if known) or simply an indicator that a certain directory scan started/ended. Keep messages small (just metadata, not file contents). Using JSON is convenient, but for high throughput a binary format (Avro, Protocol Buffers) could be used. However, JSON is fine for moderate needs and easier to debug.

Real-time Updates: The message queue enables real-time updates to users or systems. An **aggregator service** can consume the messages and update a dashboard or send notifications. For example, the aggregator (could be another Python service or even a simple script) reads messages from the queue: for progress events, it might update an in-memory status (like “Scanning folder X – 50 files scanned so far”); for file findings, it might append them to a results list or database. This aggregator can expose a web UI or API. A lightweight approach is to have the aggregator push updates via WebSockets or Server-Sent Events to a web frontend so that as messages arrive, the UI updates with new files found or progress percentage. If

building a UI is outside scope, simply logging the progress or writing to a central log file that users can tail is also an option (but less interactive).

Message Queue Deployment: Ensure the message broker is deployed in a **highly available** manner in the on-prem cluster. Kafka can be a StatefulSet with multiple brokers; RabbitMQ can be clustered; Redis can run with replication. The throughput requirements should be considered – if scanning a million files, and sending an event per file, that's a million messages in a short time. Kafka shines in that scenario ²¹, whereas RabbitMQ might need tuning (it can handle many messages but ensure persistence is appropriately configured). If message volume is very high and you only need final results, you could throttle events (e.g. not send every single file scanned, only send when an old file is found or send periodic summaries of progress). Design the schema and topics/queues such that consumers can handle the load and filter what they need (for example, separate “file_found” events from general progress so the UI can subscribe to critical events only).

Schema Example: A simple schema for a JSON message might be:

```
{
  "run_id": "2025-06-10T21:00:00Z_ABC123",
  "event": "FILE_FOUND",
  "directory": "/data/projects/foo",
  "file": "old_report.csv",
  "modification_time": "2021-05-01T12:34:56Z",
  "age_days": 769
}
```

Another for progress:

```
{
  "run_id": "2025-06-10T21:00:00Z_ABC123",
  "event": "DIRECTORY_COMPLETE",
  "directory": "/data/projects/foo",
  "files_scanned": 1500,
  "old_files_found": 42,
  "elapsed_seconds": 8
}
```

These are just examples – you can tailor the fields as needed. The key is that the aggregator knows how to interpret them. Also consider using a **schema registry** (for Kafka, if using Avro) or at least version your message formats if they evolve.

Progress Feedback: In addition to file results, it's useful to send periodic progress updates – e.g. each scanner might send a heartbeat like “still scanning X, N files processed so far” every few seconds, so the system knows it's alive. If a scanner dies unexpectedly (pod crash), the absence of a completion message or an error message can be detected by the aggregator to alert that part of the scan failed.

On-Demand and Scheduled Execution

Supporting both scheduled and on-demand execution increases flexibility:

- **Scheduled Scans with CronJob:** Kubernetes **CronJob** is the native way to run jobs on a schedule (cron syntax). We define a CronJob that wraps the parent scanner Job template. The CronJob controller will create a Job at the specified schedule (e.g. nightly at 2am) ²² ²³. Each Job then runs the scanning workflow. Best practice is to set `concurrencyPolicy` in the CronJob spec – for example, `Forbid` if you do not want a new scan to start if the previous one is still running, or `Replace` to terminate the old one if a new one begins (or `Allow` if overlapping runs are acceptable). Also set a `startingDeadlineSeconds` if needed, which is how long after the scheduled time the job can start (in case of delays or cluster issues). CronJobs are well-suited for routine scans (like daily cleanup checks). The CronJob object will manage creating Jobs, and you can limit how many successful or failed Jobs are retained (using `successfulJobsHistoryLimit` and `failedJobsHistoryLimit`).
- **On-Demand Execution:** There are several ways to trigger scans on demand:
 - Manually creating a Job (via `kubectl`) when needed, pointing it at the desired parent directory. This is the simplest: you could templatzize the Job manifest so an admin or user can run something like `kubectl create -f scanjob.yaml --env DIRECTORY=/path/to/scan`.
 - Exposing an API: You might create a small web service that when called (via REST or gRPC) will use the Kubernetes API to spawn the parent Job. For example, a “Scan Request” API could take a directory path (or choose a default set of directories) and then in the handler, instantiate the Job. This could be part of a larger system or just a thin wrapper for user convenience.
 - Using Kubernetes Operators/CRD: As mentioned, a custom resource like `FolderScan` can be defined, and an Operator (controller) watches for new `FolderScan` objects and responds by launching the scan jobs. This is more complex but integrates nicely with Kubernetes’ declarative approach (the user “creates” a `FolderScan` resource in the cluster to request a scan).
 - **KEDA (Kubernetes Event-Driven Autoscaling):** KEDA can scale a **ScaledJob** or Deployment based on external event triggers. For instance, you could configure KEDA to monitor a message queue (like a specific RabbitMQ queue or Kafka topic) for scan requests. When a message appears (indicating a new scan request), KEDA could launch a Job to handle it. However, since our focus is not on autoscaling workers from a queue trigger but rather launching an orchestrated workflow, KEDA is not a direct fit for orchestrating multiple jobs – it’s more for scaling homogeneous consumers. Still, it could be used if on-demand triggers come from a queue.

In practice, the simplest on-demand method is often the best: e.g., run the parent job manually or via a script. For user-friendliness, you might integrate it into a CI/CD pipeline or a chatbot/CLI tool that triggers scans when needed. Just ensure that concurrent on-demand scans won’t step on each other – if they might overlap in the same directories, perhaps lock those or queue requests. If different directories, they can run concurrently but will multiply the resource usage.

CronJobs and on-demand triggers can coexist. Just be mindful if an on-demand scan is running and a scheduled time hits – again, `concurrencyPolicy` in CronJob will dictate what happens. Likely you’ll set it to `Forbid` to avoid overlap.

Central Aggregation and Real-Time Visualization

Collecting all results and progress in one place is crucial for visibility. The message queue provides the transport, but we need to aggregate and present the data:

Aggregator Service: A dedicated **aggregation service** (could be a Python script, a service written in Go, or even Logstash/Fluentd depending on approach) reads from the message queue and consolidates information. This service might perform the following: - Maintain an in-memory map of current scan runs and their status (e.g., for each `run_id`, track which directories have completed scanning, how many files scanned, how many old files found). - Write important events to a persistent store. For example, when a scan completes, store a summary record in a database (like PostgreSQL or an Elasticsearch index): including total files scanned, number of old files, duration, etc. Also store the list of old files found (though for very large results, storing in a file or object storage might be better). - Provide an API or UI to query the status/results.

Real-Time Dashboard: For user visualization, a web dashboard can display: - A progress bar or status per directory being scanned (e.g. "scanning /foo/bar – 300/1000 files scanned, 5 old files found so far"). - A final summary of each directory or the whole run. - A table of old files found (with columns: file path, last modified date, age, maybe size). This dashboard can be updated in real time by hooking into the aggregator service. A simple approach is using WebSockets: the aggregator pushes events to connected clients. Alternatively, the frontend can poll the backend for updates every few seconds (simpler but slightly less real-time).

Persistent Results and Visualization: For long-term analysis, you might send results to a database or indexing service. For instance: - **Elasticsearch + Kibana:** Send each file-found event as a log entry to Elasticsearch (via Logstash or directly using Elasticsearch API). Kibana can then query and visualize, for example, list all files older than X days found in last scan, or show trends over time (did the number of old files increase or decrease compared to last week's scan?). - **Prometheus + Grafana:** More for numeric metrics than individual file lists. But you could push metrics like "old_files_found_total{directory=...} = N" via a Pushgateway, and have Grafana display time-series of counts. However, this doesn't capture file names; it's more for aggregate stats. - **Custom Web UI:** If specifics matter (file paths, etc.), a custom UI is likely needed. A lightweight approach: store results in a SQLite or CSV and provide it as a downloadable report at the end of the run.

Combining Logs and Results: It's wise to also centralize logs from the scanning pods. Use the cluster's logging solution (for example, Fluent Bit/Fluentd to ElasticSearch, or CloudWatch, etc.) so that if something goes wrong in a scanner, you can search its logs. The message events might already contain enough info, but raw logs (with stack traces on errors, etc.) are helpful for debugging. The sidecar approach can actually facilitate this – one sidecar could be a logging agent. Otherwise, ensure that `stdout` / `stderr` from all containers are collected by Kubernetes logging.

Example Visualization Workflow: The aggregator service could be as simple as a Python program using a Kafka consumer group. It reads events, updates an internal state, and prints updates or serves an HTTP endpoint for current status. For a more advanced setup, if using Kafka, you could use Kafka Streams or ksqldb to aggregate events (for example, count old files per directory in real time). Or if using RabbitMQ, the aggregator is basically the one consumer that receives everything and handles it.

The **goal** of aggregation is to *present actionable information in real time*. So design what you want to see: likely a view per scan run with a list of problematic files. You might also integrate alerts – e.g. if a very old file or a huge number of old files are found, the aggregator could trigger an alert (send email or create an issue/ticket).

Finally, once results are aggregated, consider cleaning up or resetting state between runs. If using a database, separate tables/records by run. If using just in-memory in aggregator, it should reset when a new run starts (or keep multiple runs distinct). The message queue itself might have retention (Kafka) or gets drained (RabbitMQ), so historical data should move to a more permanent home if needed.

Reliability and Resilience (Error Handling & Cleanup)

A production-grade scanner must handle errors gracefully and be resilient to failures:

- **Job Failures & Retries:** Kubernetes Jobs will retry a Pod on failure up to a limit. You can set `backoffLimit` (e.g. 3) so that if a scanner pod exits with a non-zero code, it will be retried up to 3 times ²⁴. Common causes might be a transient network error (e.g. unable to send to message queue) or a bug causing a crash. Ensure your Python code returns a non-zero exit on exceptions so that K8s knows it failed. Use try/except around scanning logic to catch errors per file (so one bad file doesn't crash the whole job). If a specific file or folder causes repeated crashes (perhaps due to encoding issues in a filename or a special filesystem glitch), you might implement a **skip list** or error tracking to skip that item after a couple attempts, rather than an infinite crash loop.
- **Timeouts:** For very large directories, a scan might hang or take too long (e.g. if a network filesystem is unresponsive). Use Kubernetes' `activeDeadlineSeconds` in the Job spec to globally time out a job if it exceeds a certain duration. For example, if a child job hasn't finished in 1 hour (which is unusual for a single directory scan, perhaps), K8s will terminate it. The parent job could also watch the time and report if a child is stuck. Additionally, within the Python code, you could use watchdog timers or timeouts for operations (though listing a directory doesn't have a built-in timeout, you'd rely on OS which might hang if NFS hangs). In practice, `activeDeadlineSeconds` is a good safety net.
- **Parent Orchestration Logic:** If the parent job is coordinating child jobs, consider having it **wait** for them to finish. This could be done by polling the Kubernetes API for Job status or Pod status. The parent can list the Jobs it created (label them with a common label for easy filtering) and periodically check if all have `status.succeeded` or if any failed. The parent might even subscribe to the message queue for "child completed" events and count them. Once all child jobs complete, the parent can send a final summary message ("All subdirectories scanned, X total files, Y old files found") and then exit successfully. If one or more child jobs failed (after retries), the parent might decide to mark the whole scan as partially failed – possibly trigger a cleanup or a follow-up scan for those failed parts, or at least log an error.
- **Recursive Spawn Considerations:** If child jobs themselves spawn grandchildren, you need a mechanism to avoid infinite recursion or uncontrolled spawning. This could be as simple as passing a "depth" parameter and decrementing it each level, or the parent only allows one level and children do no further spawning. Uncontrolled spawning could overwhelm the cluster (think a deep nested

structure creating thousands of jobs quickly). Implement guardrails: e.g., do not spawn a new job if the cluster already has N active scan jobs (the parent can check and queue the rest or wait).

- **Message Delivery Guarantees:** Design for at-least-once message delivery. This means a file-found event might be processed twice if a scanner crashed after sending it, or if a retry re-scans the same file. The aggregator or post-processing should handle duplicates (e.g. ignore if same file reported twice in the same run). Use message acknowledgments appropriately: for RabbitMQ, only ack after processing; for Kafka, commit offsets after processing. If a message queue is down or unreachable, the scanners should not crash outright – you could implement a retry logic for the message send (e.g. try a few times, or buffer locally and retry). In worst case, if the queue is down, perhaps the scanner can fall back to writing results to a file on a mounted volume so that they aren't lost, and then exit with an error (so that you know the run wasn't fully reported). The sidecar approach could help here: a sidecar could possibly cache messages until the broker is back. Tools like **Redis** as a local buffer or simply writing to stdout (with fluentd catching it) could be a fallback if direct queue publish fails.
- **Cleanup of K8s Resources:** Enable the **TTL controller** for Jobs by adding `ttlSecondsAfterFinished` in each Job spec (both parent and children) ²¹. For example, set TTL of a day or a few hours so that once the job is done, Kubernetes will automatically clean up the Job and Pod objects after that time. This prevents the cluster from accumulating thousands of completed jobs over time. Alternatively, you could have a periodic cleanup CronJob to delete jobs older than X (but TTL is easier). Also, ensure that the CronJob history limits are set reasonably (by default it keeps 3-10 old jobs; you can lower that if not needed). The actual container filesystems (images layers) will remain cached on nodes; periodically you might clean node caches if space is an issue, but generally reusing the same image for each job means the cost is paid once per node.
- **Error Logging and Alerts:** If a job fails even after retries, Kubernetes will mark it as failed. You can configure an alert (e.g. via Prometheus alert on `job_failed` metric or simply via a Slack webhook triggered by the aggregator when it sees an error event). The system operators should be notified if a scan didn't complete successfully, so it can be investigated. The aggregator service can also detect that not all expected "completion" messages were received and raise an alert.
- **Resilience to Node Failures:** If a node goes down, any running scan pods on that node will be rescheduled by the Job controller (since they'll be seen as failed). Using anti-affinity to spread jobs across nodes might help performance (spreading I/O load) and reduce impact of one node failure. But typically, just let Kubernetes handle rescheduling. If the message broker is on the same cluster, consider making it highly available across nodes too (so it's not a single point of failure).
- **Retries for Scan Errors:** On the application level, if a particular directory scan fails (maybe due to a rare error), you could allow the parent to retry that subdirectory by spawning a new job for it (maybe after some delay). This is essentially what K8s Job retry does at pod level. But if a failure is deterministic (like "permission denied" on a directory), a retry will not fix it. So such errors should be caught and handled: e.g., log and skip that directory (and perhaps report it as an error in results). It's better to continue the overall scan than to halt everything for one inaccessible folder.

In summary, **design for graceful degradation**: One failing component (one directory's scan) should not break the entire workflow. Use timeouts, retries, and fallbacks to ensure the scan completes as much as possible, and report clearly any sections that were skipped or failed.

Security, Logging, and Monitoring Best Practices

Security Considerations: Running numerous containers that access potentially sensitive file paths requires careful security controls:

- **Least Privilege:** As noted, run containers as non-root whenever possible ¹⁸. If the scan requires elevated permissions (for example, scanning system directories on a node), consider alternatives like using a DaemonSet on each node with hostPath (which is more complex and risky) or mounting the volume with appropriate permissions. Ideally, the scanning is for user data directories where a non-root user with access is feasible.
- **Read-Only Filesystem:** Mount target directories as read-only if possible. The scanning job only needs to read metadata (and possibly file contents if the use-case needed it, but here only timestamps). Using a read-only mount prevents any accidental modification. Also, mount the container's root filesystem as read-only if the app doesn't need to write (this can be set in the pod security context) – it adds protection against certain attacks.
- **Network Policies:** If using a cluster network that supports NetworkPolicies, restrict the scanning pods so they can only communicate with the message queue and not anything else. They likely don't need to call external services except the broker; lock that down to limit impact if compromised.
- **RBAC:** The parent job's service account should have permission only to create child jobs (and maybe to list pods/jobs for monitoring). It should not have broad cluster permissions. Use a dedicated Kubernetes Role for this.
- **Image Security:** Scan your container images for vulnerabilities (using tools like Trivy or Clair) as part of the CI/CD. Keep the base images updated ²⁵. Since these scan containers might run on a schedule, you don't want them to be the source of a security breach over time.
- **Secrets Management:** If credentials are needed (for example, to authenticate to the message queue or to some API), use Kubernetes Secrets and mount them as env vars or files, rather than baking into images. Also, consider rotating these if possible.
- **File Data Sensitivity:** We are sending file paths and metadata over the message bus. If that data is sensitive (contains user names, project names, etc.), ensure the message channel is not accessible by unauthorized parties. Use encryption on the wire (e.g., TLS for Kafka or RabbitMQ) and at rest if stored. Possibly mask or omit extremely sensitive info (like file content – which we are not sending, just meta).
- **Pod Security:** Apply Pod Security Standards (baseline at least). The containers should drop unnecessary Linux capabilities (e.g., they likely need none beyond default, so consider `drop: ["ALL"]` and add only if needed). Also, set `readOnlyRootFilesystem: true` in securityContext if possible, and do not allow privilege escalation.
- **Isolation:** If scanning untrusted file paths (like user-uploaded files), be aware of possible malicious inputs (e.g., a file path that is extremely long or has strange chars might break naive code). Use safe Python practices (maybe use `os.walk` which handles recursion safely and catches errors, or wrap calls in try/except).

Logging: As mentioned, log to stdout/stderr so that Kubernetes can capture the logs ²⁶. Structure log messages with context (include the directory or file being processed in the log line) to aid debugging. Use a consistent format (maybe JSON logging or key-value) so that if logs are shipped to Elastic, you can search easily. For progress, relying on the message queue is fine, but having logs is a good fallback. If using sidecar for logging (like some setups have a sidecar running something like Fluent Bit), ensure it doesn't introduce too much overhead.

Monitoring: Deploy monitoring for this system on multiple levels:

- **Kubernetes Monitoring:** Use Prometheus to scrape metrics from Kubernetes about Jobs/Pods. For example, Kube-state-metrics can provide metrics like `kube_job_status_succeeded{job-name=...}` and `kube_job_status_failed`.

This can be graphed or alerted on (e.g., alert if any scan job fails). Also monitor resource usage: if scanning jobs occasionally get OOMKilled (out-of-memory), you'd see that in Pod metrics or events. - **Application Metrics:** Integrate metrics in your Python code. For instance, use the Prometheus client library to count files scanned, or duration of scans, and either push these metrics (the jobs are short-lived, so you can't reliably scrape them – instead use a Pushgateway or have them send metrics in their final message). Alternatively, the aggregator can accumulate metrics across runs and expose them. Key metrics might include: number of files scanned per run, number of old files found per run, time taken per directory, etc. Over time, you can observe trends (maybe the number of old files is growing, etc.). - **Message Queue Monitoring:** Monitor the health of the message queue (broker uptime, queue lengths). If a queue backlog grows unexpectedly, that might indicate consumers (aggregator) are down or slow. If using Kafka, monitor offsets lag. - **Distributed Tracing (optional):** For a complex system, one could use tracing to follow a scan request as it triggers various jobs and messages. This might be overkill for a file scanner, but if needed, you could label messages with a trace ID and use something like OpenTelemetry.

Example Best Practices Checklist: - Run containers with minimal privileges (non-root user, limited FS access) ¹⁸. - Keep Docker images updated and small ²⁷. - Use Kubernetes features like CronJob for scheduling and Jobs for robust retry logic instead of reinventing scheduling. - Use sidecars only if needed (for logging or special tasks) and take advantage of native sidecar support in new K8s versions to avoid custom teardown logic ⁵. - Implement thorough logging and send significant events to the message queue for real-time monitoring. - Test failure scenarios: simulate a child pod crash, message broker outage, etc., and verify the system handles it (retries or at least fails cleanly without stalling other components).

By following these practices, the folder scanning system will be robust, efficient, and maintainable, providing timely information on old files across large directory structures in an on-premises Kubernetes environment.

¹ ² **Parallel Processing using Expansions | Kubernetes**

<https://kubernetes.io/docs/tasks/job/parallel-processing-expansion/>

³ ⁴ **Fine Parallel Processing Using a Work Queue | Kubernetes**

<https://kubernetes.io/docs/tasks/job/fine-parallel-processing-work-queue/>

⁵ ⁶ ⁷ ⁸ ⁹ **Properly Running Kubernetes Jobs with Sidecars in 2024 (K8s 1.28+) | by Steven Aldinger | TeamSnap Engineering | Medium**

<https://medium.com/teamsnap-engineering/properly-running-kubernetes-jobs-with-sidecars-in-2024-k8s-1-28-ad9b51d17d50>

¹⁰ ¹¹ ²¹ **secureCodeBox - Continuous Security Testing | ITNEXT**

<https://itnext.io/exploring-securecodebox-an-open-source-continuous-security-testing-solution-for-devsecops-b233fc5341e1?gi=d6a825bac0da>

¹² ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²⁵ ²⁶ ²⁷ **Docker Best Practices for Python Developers | TestDriven.io**

<https://testdriven.io/blog/docker-best-practices/>

²² ²³ **The Ultimate Kubernetes CronJob Guide**

<https://cronitor.io/guides/kubernetes-cron-jobs>

²⁴ **Kubernetes Vulnerability Scanning: What You Should Know**

<https://www.aquasec.com/cloud-native-academy/kubernetes-101/kubernetes-vulnerability-scanning/>